

Pandas, Data Cleaning & EDA Common Practices

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 1, Day 06, Self-study, Olympiad practice

24 September 2026

1. Where This Fits
2. Learning Outcomes
3. What a Tabular Task Looks Like
4. Common Tricks
5. Common Mistakes
6. Tips
7. Common Practices
8. Olympiad Practice
9. Check Yourself
10. Limitations and What Comes Next
11. Summary
12. References

- ▶ **This week gave you the tools:** Python and loops (Day 01), vectors and NumPy arrays (Day 02, Day 05), pandas, cleaning and plots (Day 03), classes with `fit` and `predict` (Day 04).
- ▶ **This deck is about using them under competition conditions:** a table you have never seen, a clock, a submission file that is either accepted or worth zero. Every frame is one move, taken from a real Olympiad task, that wins points or loses them.
- ▶ **How to use it:** one item per frame: the situation, then the action. Keep the checklist frame open whenever you meet a new table. Nothing here is new; it is the week's tools in the order a competitor uses them.

Real Olympiad tables are not the Iris table

- ▶ **HAIO 2025 online qualifier (Hungary)**: 1,000 applicants, 31 columns, predict admission. Exam scores use **−1 for “did not take the subject”**; `isna()` reports nothing, yet a third of some columns is missing. Treat **−1** as a score and every mean is wrong.
- ▶ **Adult census (a classic in national rounds)**: 32,561 rows, `df.isna().sum()` is 0 everywhere, but 1,836 rows of `workclass` contain the text `"?"`.
- ▶ **NEOAI 2025 “Tricy Table”**: 12 feature columns, every one with thousands of missing values, and a **frozen model**: the organisers fixed the training function and let contestants change only the table. Baseline 3.14, winner 2.24 (lower is better). The whole gap came from data work.

After this deck you can

Find what is wrong with a table in ten minutes, fix it the same way on train and test, produce a submission that is accepted on the first try, and know which mistakes cost the most points.

By the end of this deck, you will be able to:

- ▶ **Run** a ten-minute EDA routine on an unknown table and write down its five key numbers.
- ▶ **Find** missing values that hide as "?", -1 or 0, impossible values, and duplicated rows or ids.
- ▶ **Clean** train and test with one function so both tables get the same treatment.
- ▶ **Avoid** the five mistakes that turn a correct model into a zero: wrong row count, wrong format, leaked ids, mismatched cleaning, filling with the wrong value.
- ▶ **Build** the submission file first, from a baseline, and check it against the sample before improving anything.
- ▶ **Recognise** the tabular task format: train, test, sample submission, one metric.

Pandas: **What a Tabular Task Looks Like**

The Shape of a Tabular Olympiad Task

You get	What it is	Example (HAIO 2025 qualifier)
train.csv	rows with features and the target	1,000 applicants, 31 columns, target admitted (1 or 0)
test.csv	the same features, no target	the applicants to predict, each with an ID
sample_submission.csv	the exact file format expected	ID, admitted, one line per test row
a metric	how the file is scored	ROC-AUC; often accuracy
a baseline	the score you must beat	logistic regression on one column (school prestige)

- ▶ The organisers keep the test labels. Your file is compared with them: every test row needs exactly one answer.
- ▶ **Many national rounds add questions about the table itself** before the model, for example “the floor of the mean `Monthly_Inhand_Salary` where `Credit_Utilization_Ratio` ≥ 25 ” (answer 4106) or “how many distinct values of `Month`” (8), handed in as a small answers file. The same tools answer both kinds of task.

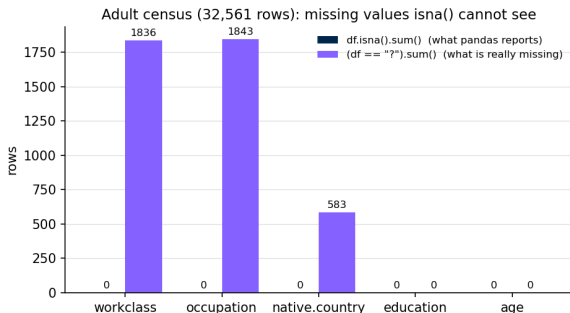
Run these in this order on every new table. Write the answers in a text cell; you will need them later.

```
train = pd.read_csv("train.csv"); test = pd.read_csv("test.csv")
train.shape, test.shape           # 1. rows and columns of both
train.head(3)                     # 2. what a row looks like
train.info()                       # 3. types: text that should be numbers?
train.isna().sum()                 # 4. missing, part one
(train == -1).sum()                # 4b. missing, part two (see Tricks)
(train == "?").sum()
train.describe()                   # 5. min and max: impossible values
train.duplicated().sum()           # 6. repeated rows
train["target"].value_counts(normalize=True) # 7. class balance
```

Seven cells, no thinking required, and they catch most of what the next frames describe. The five numbers to write down: rows in train, rows in test, missing per column, duplicates, share of each class.

Pandas: **Common Tricks**

Trick: Missing Values Do Not Always Look Missing



Source: `scripts/day06_eda_figures.py`, Adult census (Kaggle `uciml/adult-census-income`).

When it applies

`isna().sum()` says 0 but `info()` shows a column as object that should be numeric, or a category has a value like "?", "unknown", -1, 999, 0.

What to do

For every text column run `value_counts()`; for every numeric column look at `min` and `max` in `describe()`. Then `df.replace("?", np.nan)` and treat it like any other missing value.

Trick: value_counts on Every Text Column

```
for col in train.select_dtypes("object").columns: # text columns
    print(col, train[col].nunique())
    print(train[col].value_counts(dropna=False).head(6), "\n")
```

- ▶ **Adult census:** native.country shows United-States 29170, Mexico 643, ? 583: the third most common “country” is a question mark.
- ▶ **NOAI Singapore 2025 (mental-health table, 98,490 rows):** Sleep Duration has 30 values: four real ones with 22,000 to 27,000 rows each, then 26 junk ones with 1 to 8 rows ("Pune", "45", "Sleep_Duration"). Keep the allowed set, turn the rest into NaN.
- ▶ **Cram School round (fitness table):** smokes holds yes 565, 0 473, no 416, 1 146: two encodings in one column. map({"yes": 1, "1": 1, "no": 0, "0": 0}); the published solution mapped only "yes" and lost 146 smokers.
- ▶ dropna=False makes real NaN appear in the count too, so the two kinds of missing sit side by side.
- ▶ Also caught: a column with one value per row (an id in disguise), and a target with three classes when you expected two.

Trick: A Number Column That Is Text: Look at Its Characters

```
train["Age"].dtype           # object      (Credit Score, OJIA 2025)
train["Age"].unique()[ :6]   # ['23' '28_' '34' '2373_' '54' '21']
"".join(sorted(set("".join(train["Age"])))) # '0123456789_' <- the culprit
train["Age"] = train["Age"].str.replace("_", "").astype(float)
train["Age"].describe()      # min -500, max 8697: next trick
```

When it applies

`info()` shows object for a column that should be numeric, and
`to_numeric(errors="coerce")` would turn thousands of rows into NaN.

What to do

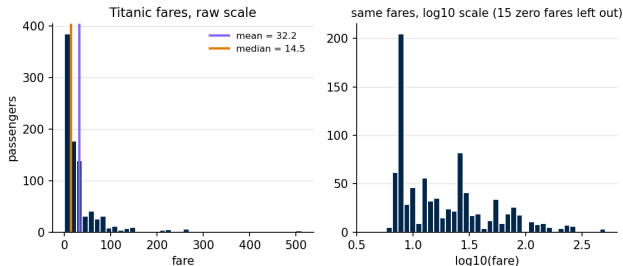
Print the **set of characters** the column uses (one line above): a stray `_`, a double dot
("102..`5`" in a powerlifting table), a unit. Remove that one character, then convert. Nothing
is lost. **The contrast:** the organisers' own beginner solution used
`to_numeric(errors="coerce")` and `dropna()`: 10,766 of 44,207 rows gone, 24% of the
table, for one underscore.

Trick: describe() Is an Outlier Detector

```
train.describe().loc[["min", "50%", "mean", "max"]]  
#      age  capital.gain  hours.per.week  Fare (Titanic)  
# min    17           0.0           1.0      0.00  <- free tickets?  
# 50%    37           0.0          40.0      14.45  
# mean   38.6        1077.6          40.4      32.20  <- 2 x median  
# max    90        99999.0          99.0     512.33
```

- ▶ **A max that is a round number:** capital.gain = 99999 in 159 Adult rows is a cap the collectors typed in, not money. hours.per.week = 99 in 85 rows, same story.
- ▶ **A min of 0 in a column that cannot be 0:** 15 Titanic fares are 0. A missing fare written as zero.
- ▶ **Mean far from median:** the column is skewed (next frame). Use the median when you fill it.
- ▶ Ask “can this value exist?” for every min and max. It takes one minute and finds what `isna()` never will.

Trick: When Mean and Median Disagree, Trust the Median



Source: scripts/day06_eda_figures.py, Titanic (Kaggle yasserh/titanic-dataset).

When it applies

Money, counts, durations: a long tail of large values. Mean 32.2 versus median 14.5 here.

What to do

Fill missing values with `median()`, compare groups with median as well as mean, and plot on a log scale to see the shape. Week 2: models like the log of such columns.

Trick: groupby Against the Target Tells You What Matters

```
train.groupby("Sex")["Survived"].mean()      # female 0.74, male 0.19
train.groupby("Pclass")["Survived"].mean()  # 0.63, 0.47, 0.24
rich = adult["income"] == ">50K"
rich.groupby(adult["education"]).mean().sort_values()
# ... Masters 0.56, Prof-school 0.73, Doctorate 0.74
train.corr(numeric_only=True)["Survived"].sort_values() # all numeric
```

- ▶ One line per column, sorted: the columns whose groups differ most are the ones a model will use. The HAIO sample solution did exactly this before choosing features.
- ▶ It is also the **one-rule baseline**: “predict survived if female” scores 0.79 on Titanic, above the 0.62 majority baseline, with no model at all.
- ▶ Where you meet it: “the mean of X per group” is the most common question about a table, and this is the answer.

Pandas: **Common Mistakes**

Mistake: Filling With the Wrong Value

```
titanic["Age"].mean()           # 29.7   (177 of 891 missing)
titanic["Age"].fillna(0).mean()  # 23.8   <- 177 newborns invented
median = titanic["Age"].median() # 28.0
titanic["Age"].fillna(median).mean() # 29.4
haio["math_exam"].mean()        # wrong: -1 = "did not take it"
haio["math_exam"].replace(-1, np.nan).mean() # right
```

Symptom

A mean that moved a lot after filling, or a column whose minimum is a code (-1, 0, 999) rather than a measurement. In HAIIO 2025 the sample solution counted `(train == -1).sum()` next to `isnull().sum()`: two kinds of missing, counted separately (`math_exam` stands for its Hungarian column name).

Fix

Turn codes into NaN first (`replace`), then fill with the median of the column, and keep a flag column `col_missing` if “was it missing” might carry information (it did not in Tricky Table; it does for “did not take the exam”).

```
df.groupby("Working Professional or Student")["Academic Pressure"].apply(  
    lambda s: s.isna().mean())      # Student 0.00, Working Professional 1.00  
# students have Academic Pressure, professionals have Work Pressure  
df["Pressure"] = df["Academic Pressure"].fillna(df["Work Pressure"])
```

Symptom

A column is missing for exactly the rows of one group and present for the other. **NOAI Singapore 2025:** Academic Pressure is NaN for 100% of working professionals, Work Pressure for 100% of students. The organisers' baseline median-filled both; the values it invented mean nothing.

Fix

Ask “why is it missing?” with a groupby on the missing mask before filling. Two columns that are never both present are one column: combine them. **Another real case (OJIA 2025, calories):** 500 test rows had Gender missing on purpose; the brief said they were “a male handball team”. The answer was in the text, not in `fillna`.

```
med_train = train["Age"].median() # 28
train["Age"] = train["Age"].fillna(med_train)
test["Age"] = test["Age"].fillna(test["Age"].median()) # 27, other
test = test.dropna() # <- rows vanish: file too short
```

Symptom

A good local score and a bad leaderboard score, or a rejected submission because the file has fewer rows than `sample_submission.csv`.

Fix

Compute every fill value, every category list and every scaling number **on train**, then apply the same numbers to test. Never drop a test row: fill it instead. The Tricky Table solutions all did `imputer.fit_transform(train)` then `imputer.transform(test[train.columns])`: fit once, apply twice, same column order.

Mistake: The Id Is Not a Feature, the Target Is Not Either

```
X = train.drop(columns=["ID", "admitted"])    # features only  
y = train["admitted"]                        # the target, kept apart  
ids = test["ID"]                             # kept for the submission
```

Symptom

A model that scores almost perfectly on train and badly on test, or a submission whose rows are in the wrong order.

- ▶ The ID is a row number. If the organisers sorted the file by outcome before numbering it, the id “predicts” the target on train and nothing on test. The HAO solution’s first modelling line is “separate the target and the ID”.
- ▶ A column that is the target in disguise (education.num next to education; a “refund” column next to “churned”) gives the same false comfort. Ask of every column: **would I know this before knowing the answer?**
- ▶ test.csv never has the target column. If your cleaning function refers to it, it fails on test.

Mistake: A Submission That Is Not the Sample's Shape

```
sample = pd.read_csv("sample_submission.csv")
sub = pd.DataFrame({"ID": test["ID"], "admitted": preds})
sub.to_csv("submission.csv", index=False)      # index=False, always

check = pd.read_csv("submission.csv")
assert list(check.columns) == list(sample.columns), "column names"
assert len(check) == len(sample), "one row per test row"
assert (check["ID"] == sample["ID"]).all(), "same ids, same order"
```

The most expensive mistake in any competition

A wrong column name, an extra unnamed index column, 889 rows instead of 891, or probabilities where labels were expected: the file scores zero or is refused, and you find out after the deadline. HAIO 2025 even prescribed the file name (HUN123_1.csv); NEOAI's frozen function wrote the file itself so nobody could get it wrong.

Four asserts, ten seconds, every time you write the file. And one more from the Romanian judge: answers are compared **as text**, so 54.0 is not 54. Cast a count to int before writing it.

Pandas: **Tips**

Tip: Submit Something in the First Fifteen Minutes

```
majority = train["admitted"].mode()[0]          # the most common class
sub = pd.DataFrame({"ID": test["ID"], "admitted": majority})
sub.to_csv("submission_0.csv", index=False)      # valid file, known score
```

Situation

The clock is running, the table is dirty, and you have not decided what to do.

Do this

Build the submission pipeline before any cleaning: read, predict the majority class (or the train mean for a number), write, check the shape. Now every later step is an improvement on a file that already scores. The HAO baseline was one column into logistic regression; Titanic's majority class alone is 0.616. Day 04's MajorityClassifier is this tip as a class.

Tip: One Cleaning Function, Applied to Both Tables

```
def clean(df, fill_values):  
    df = df.replace({"?": np.nan, -1: np.nan})           # hidden missing  
    df = df.drop(columns=["Cabin"], errors="ignore")     # ok on test too  
    for col, value in fill_values.items():  
        df[col] = df[col].fillna(value)                 # train's numbers  
    df["Sex"] = (df["Sex"] == "female").astype(int)     # text to 0/1  
    return df
```

```
fills = {"Age": train["Age"].median(), "Fare": train["Fare"].median()}  
train_c, test_c = clean(train, fills), clean(test, fills)
```

- ▶ Day 01's rule, "if you paste it twice, make it a function", is worth points here: the two tables cannot drift apart.
- ▶ The NEOAI solution author wrote: the main code "is in the clean_df function". That is where the 3.14 became 2.95.
- ▶ Print `train_c.shape`, `test_c.shape` right after: same columns, original row counts.

Tip: Score Yourself the Way the Organisers Will

```
from sklearn.model_selection import train_test_split
tr, va = train_test_split(train_c, test_size=0.2, random_state=0,
                           stratify=train_c["admitted"])
# fit on tr, predict va, then compute THE COMPETITION METRIC
# on va, in the original units
```

Situation

You cannot see the test labels, and you get few submissions per day. You need a number you can trust before you submit.

Do this

Hold out 20% of train (stratified, so both parts have the same class balance), and compute the exact metric from the task page on it. **A real slip from NEOAI 2025:** a strong competitor validated in *log space* and with the wrong root in the formula; the local number (0.36) had nothing to do with the leaderboard (2.95). Same metric, same units, same shape as the submission.

Skipped on purpose: rows that belong to groups (6,247 of 6,291 test customers in the OJIA credit table also appear in train) make a random split flatter you. Day 13.

Pandas: **Common Practices**

Practice: The Notebook Has a Fixed Order

Block	Cells	Time (2-hour task)
1. Load and look	<code>read_csv</code> , <code>shape</code> , <code>head</code> , <code>info</code> , <code>describe</code>	10 min
2. Find the traps	<code>isna</code> , <code>== -1</code> , <code>value_counts</code> , <code>duplicated</code> , <code>min/max</code>	10 min
3. Baseline submission	majority class or mean, <code>to_csv</code> , the four asserts	10 min
4. Clean	one <code>clean()</code> function, both tables, shapes printed	20 min
5. Ask the data	<code>groupby</code> against the target, two plots	15 min
6. Improve and resubmit	better rule or model; hold-out score before each submission	rest

- ▶ Blocks 1 to 3 are the same on every task and take half an hour. Experienced competitors type them without thinking; make them a habit.
- ▶ Everything in this deck lives in blocks 2 to 4. Weeks 2 to 6 change only block 6.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
X_train = imputer.fit_transform(train_c)          # learns medians from train
X_test = imputer.transform(test_c[train_c.columns]) # applies them
```

- ▶ fit learns numbers from a table; transform applies them. Day 04's fit/predict shape again, for cleaning tools. Every scikit-learn cleaner (imputer, scaler, encoder) works this way; in Week 2 you will see the pair everywhere.
- ▶ test_c[train_c.columns] forces the test columns into the train order. Both NEOAI solutions did this; a swapped column pair is silent and fatal.
- ▶ Median imputation leaves a fingerprint: in the NEOAI table, several rows share the value 2563.308 in feat_1, the fill. Harmless, but do not mistake it for real data when you look at head() afterwards.

Practice: Let the Data Say How Much to Trust Each Row

```
features = train.drop(columns="target")  
missing_frac = features.isna().mean(axis=1) # share missing per row  
weight = 1.0 - missing_frac # half missing = half weight  
n_missing = features.isna().sum(axis=1) # or a new feature column
```

- ▶ `axis=1` (Day 05) turns the missing-value table into one number per row.
- ▶ **NEOAI 2025:** the frozen training function accepted per-row weights. The 2.7-score solution was the only one that used them, with exactly the line above, and beat the community solution that did fancier feature work (2.95).
- ▶ A count of missing cells per row is also an honest feature: “incomplete applications” can behave differently. Add it, check it with `groupby` against the target, keep it if it moves.
- ▶ The idea generalises: whenever the task lets you pass weights, quality of the row is the first thing to encode.

Pandas: Olympiad Practice

Task

Load `adult.csv` (Kaggle `uciml/adult-census-income`, 32,561 rows, 15 columns, target `income`). Answer:

1. How many rows remain after removing duplicated rows?
2. How many of those rows have a missing value in at least one column? (Missing values are written as "?".)
3. What share of rows has `income == ">50K"`? (3 decimals)
4. What is the mean `hours.per.week` of the `>50K` rows? (1 decimal)
5. What is the most common `workclass`, once "?" counts as missing?

Hand in `answers.csv` with columns `question`, `answer` and exactly five rows. Scoring: one point per exact answer.

Format: the `questions-about-a-table` subtasks of national rounds. Data and traps: the Adult census, a staple of tabular tasks.

Recognise: blocks 1, 2 and 5 of the notebook order; the "?" trick; groupby against the target. **Check:** the answers are 32537, 2398, 0.241, 45.5 and Private; the file has 5 rows and 2 columns.

```
df = pd.read_csv("adult.csv").drop_duplicates()      # 24 duplicates gone
a1 = len(df)                                         # 32537
df = df.replace("?", np.nan)                        # hidden missing -> NaN
a2 = int(df.isna().any(axis=1).sum())               # 2398 rows
a3 = round((df["income"] == ">50K").mean(), 3)       # 0.241
hours = df.groupby("income")["hours.per.week"].mean()
a4 = round(hours[">50K"], 1)                        # 45.5
a5 = df["workclass"].mode()[0]                     # 'Private'
answers = pd.DataFrame({"question": [1, 2, 3, 4, 5],
                        "answer": [a1, a2, a3, a4, a5]})
answers.to_csv("answers.csv", index=False)
assert len(pd.read_csv("answers.csv")) == 5
```

Order matters: deduplicate before counting, replace "?" before isna and before mode. Skip the replace and question 5 still says Private, but question 2 says 0.

Pandas: **Check Yourself**

Answer on paper before turning the page.

1. A column `exam_score` has `isna().sum() == 0`, `min == -1` and `max == 100`. What are the two things you do before computing its mean, and in which order?
2. You fill missing ages in `train` with 28 and in `test` with 27, then drop the two test rows that still have a missing port. Name the two mistakes and what each one costs.
3. A task gives you `sample_submission.csv` with columns `ID`, `answer` and 300 rows. Your file has columns `Unnamed: 0`, `ID`, `answer` and 300 rows. What went wrong, what is the one-word fix, and which assert would have caught it?

1. First `replace(-1, np.nan)` (turn the code into a real missing value), then `fillna(median)` or leave it missing; only then `mean()`. Filling before replacing would keep the `-1`s inside the median.
2. Different fill values on the two tables (the test rows are treated by a rule the model never saw; use train's 28 for both), and dropping test rows (the submission is now two rows short and is rejected or scores those rows as wrong). Fill test rows, never drop them.
3. `to_csv` wrote the index as a first column. The fix is `index=False`. The check `list(check.columns) == list(sample.columns)` fails immediately; the row-count assert would have passed and hidden it.

Pandas: **Limitations & What Comes Next**

What this deck cannot do for you

- ▶ **No checklist finds every trap.** Tricky Table's authors promised "corrupted" data; three solutions found only missing values. Look at the table with your own questions, not only with these cells.
- ▶ **Cleaning has no score of its own.** You only learn whether a fill or a drop helped by scoring a baseline before and after (Tip 3). Keep the before number.
- ▶ **More cleaning is not always better.** In Tricky Table the "is missing" flags correlated about 0 with the target; the simple median fill plus a log target did most of the work.
- ▶ **Tomorrow, Friday:** the Coursera exam, then the Week 1 lab exercise on a real table. Everything in this deck applies.
- ▶ **Week 2:** block 6 of the notebook fills up: linear models, trees, the metrics behind ROC-AUC, and `fit/transform` tools like `StandardScaler` and encoders that replace the hand-written `clean()`.
- ▶ **Skipped on purpose:** dates and text columns (Week 6), feature engineering (Day 12), and the class-imbalance tricks that HAIO's stratified split hinted at (Day 13).

Pandas: Summary

- ▶ **The ten-minute routine:** shape, head, info, isna, the hidden-missing checks, describe, duplicated, class balance. Write the five numbers down.
- ▶ **Tricks:** missing values hide as "?", -1 and 0; `value_counts` on every text column; describe min and max as an outlier detector; median over mean on skewed columns; groupby against the target to see what matters.
- ▶ **Mistakes:** filling with the wrong value, cleaning train and test differently, using the id or a leaked column as a feature, a submission that is not the sample's shape.
- ▶ **Tips:** submit a baseline in the first fifteen minutes; one `clean()` function for both tables; validate with the competition's own metric in its own units.
- ▶ **Practices:** the notebook order; fit on train, transform test, same columns; encode row quality as weights or a feature.
- ▶ In NEOAI 2025 the model was frozen and the data work alone moved the score from 3.14 to 2.24. The table is the task.

Pandas: References & Resources

- [1] NEOAI 2025 (Northern Eurasia Olympiad in AI). *Tricy Table Data* competition on Kaggle, with the organisers' baseline kernel (task text, the frozen `clf_train` function, metric $(\text{MSE})^{1/4}$).
<https://www.kaggle.com/competitions/neoai-2025-tricy-table-data>
- [2] Sandei, S. (2025). *roai-solved*, NEOAI 2025 Tricy Table solution (score 2.95; the `clean_df` function, median imputation, log target). <https://github.com/stefanasandei/roai-solved/tree/main/international-contests/neoai/2025>
- [3] *IOAI-prep-2026*, NEOAI 2025 Tricy Table solution with per-row weights (score 2.7).
<https://github.com/alwaysneedhelp/IOAI-prep-2026>
- [4] Hungarian AI Olympiad (2025). *Summer online qualifier: University Admissions*, starter notebook and sample solution (English versions).
<https://github.com/Hungarian-AI-Olympiad/HAIO-Hungarian-AI-Olympiad/tree/main/2025/nyari-online>

- [5] Becker, B. and Kohavi, R. (1996). *Adult* data set, Kaggle copy [uciml/adult-census-income](https://www.kaggle.com/datasets/uciml/adult-census-income); the "?" and 99999 examples.
<https://www.kaggle.com/datasets/uciml/adult-census-income>
- [6] Kaggle (2021). *Titanic Dataset* (yasserh/titanic-dataset); the fare and age examples. <https://www.kaggle.com/datasets/yasserh/titanic-dataset>
- [7] Olimpiada Nationala de Inteligenta Artificiala, Romania (2025). *Simulare OJIA 1: Credit Score* task and editorial (<https://github.com/Olimpiada-AI/Editoriale>); solved copies with the data in <https://github.com/jaredliw/ioai-tsp-2025> (onia-romania-2025/).
- [8] NOAI Singapore (2025). *Mental Health ML Task* (train 98,490 rows), in <https://github.com/jaredliw/ioai-tsp-2025> (noai-singapore-2025/).

- [9] Sandei, S. (2025). *roai-solved*, Cram School Educational 1 (fitness and powerlifting tables). <https://github.com/stefanasandei/roai-solved/tree/main/contests/bcs-edu-01>

- [10] The pandas development team (2026). *Working with missing data*. https://pandas.pydata.org/docs/user_guide/missing_data.html